

Homework — 1.1.2 Types of Processor — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Part A (14 marks)

1. [3] — 2 correct cells per mark: (a) **large set of complex instructions**; (b) **fixed length**; (c) **usually one cycle**; (d) **in the software/compiler**; (e) **longer programs, more RAM needed**; (f) **higher**.
2. [3] — 2 correct rows per mark: Multicore processor = **D**; Parallel processing = **B**; GPU = **C**; Co-processor = **A**; Pipelining = **F**; RISC = **E**. (*G and H are distractors: Von Neumann architecture and cache.*)
3. [3] — 1 mark per statement correctly fixed:
- 1: A GPU contains a **very large number of simpler cores** — far **more** than a typical CPU.
 - 2: Each core carries out the **same instruction on a different item of data** at the same time.
 - 3: GPUs are increasingly used for **general purpose (non-graphics) computing**, e.g. machine learning, scientific modelling/simulation, data mining.
4. [2] — 1 mark per valid reason (max 2), linked to a phone: RISC uses **less power** → longer battery life; simpler instructions run efficiently/cool in a small device without a fan; easier to pipeline → good performance per watt; cheaper/simpler hardware keeps the handset cost down. (*Generic "RISC is simpler" with no link caps at 1.*)
5. [3] — 1 mark per row (correct verdict; for false statements the correction is required):

Statement	True / False	Correction
More cores = automatically faster	False	The software must be written to use multiple cores / the task must be capable of being split (parallelised); otherwise extra cores give little or no benefit
Co-processor takes over specific tasks	True	—
Parallel processing is strictly sequential	False	In parallel processing several instructions are processed simultaneously (strictly one-after-another is serial processing)

Part B (6 marks)

6. [3] — 1 mark: Von Neumann uses a **single shared memory and bus** for both data and instructions. 1 mark: so it **cannot fetch an instruction and read/write data at the same time** (they compete for the one bus) — the bottleneck. 1 mark: Harvard uses **separate memories and buses** for data and instructions, so instruction fetch and data access can happen **simultaneously**.
7. [2] — 1 mark each (factor + effect), max 2: **clock speed** — more pulses/sec → more FDE cycles/sec; **cache size** — more frequently used data/instructions held near the CPU → fewer slow RAM fetches;

pipelining — overlaps FDE stages → higher throughput. (*Accept word/bus width.*)

8. [1] — **CIR** (Current Instruction Register).

Total: $14 + 6 = 20$